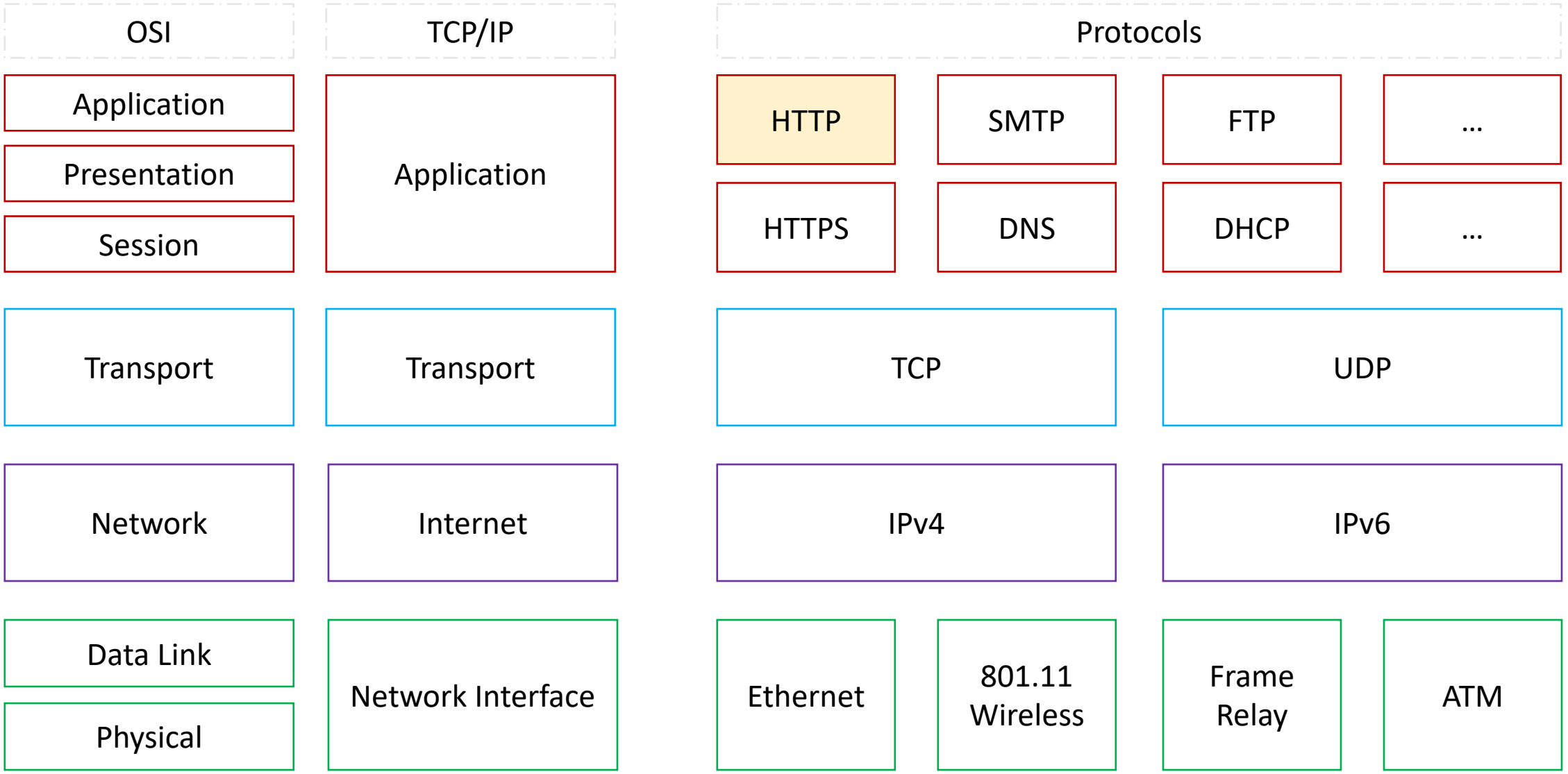
HTTP

Hypertext Transfer Protocol

Network Protocols



TCP/IP - Application Layer Protocols

- The application layer contains the ***communications protocols*** used in ***process-to-process*** communications across a computer network.

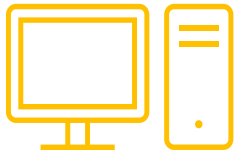
- The application layer ***only standardizes communication*** and depends upon the underlying transport layer protocols to establish host-to-host data transfer channels and manage the data exchange in a client-server or peer-to-peer networking model.

- The application layer ***does not describe specific rules or data formats*** that applications must consider when communicating.

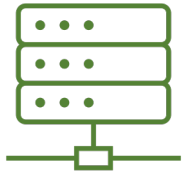
Web Services

Web App

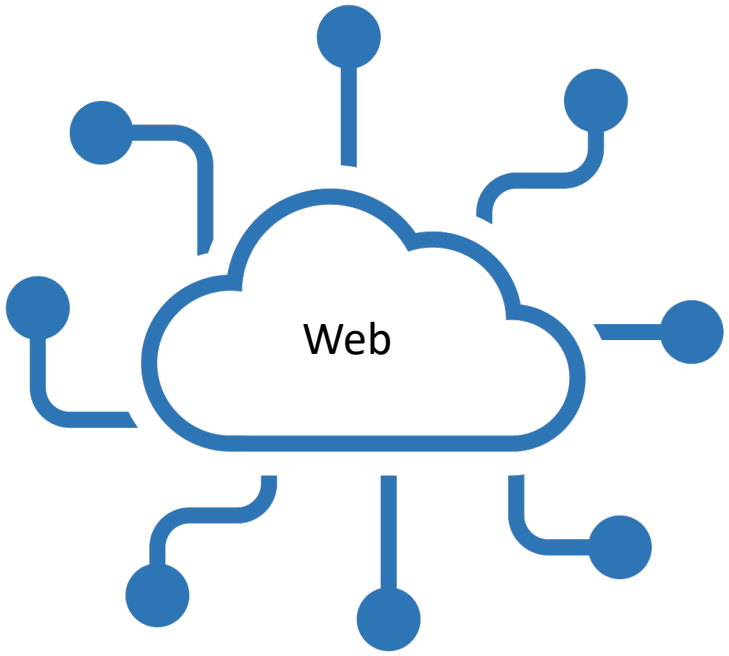
Mobile

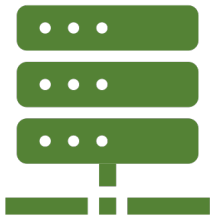


Desktop



Service





Service

HTTP

- Hypertext Transfer Protocol (HTTP) is an application-layer protocol for transmitting hypermedia documents, such as HTML.

- It **was designed for** communication between web browsers and web servers, but it **can also be used for** other purposes.

- HTTP follows a classical client-server model, with a client opening a connection to make a request, then waiting until it receives a response.

- HTTP is a stateless protocol, meaning that the server does not keep any data (state) between two requests.

HTTP Request

```
METHOD URL_PATH HTTP_VERSION
Key: value

Request Body, If Exists !!!
```

- Request Line: METHOD + URL_PATH + HTTP_VERSION

- Methods: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS, TRACE
- Response Headers: are key-value pair separated by a Colon “ : ”
- A Blank Line (CR+LF) separates between the Request Headers and Request Body

HTTP Response

```
HTTP_VERSION STATUS_CODE STATUS_MESSAGE
Key: value

Response Body, If Exists !!!
```

- Response Line: HTTP_VERSION + STATUS_CODE + STATUS_MESSAGE

- Status Code & Message: provide information about the processed Request
- Response Headers: are key-value pair separated by a Colon “ : ”
- A Blank Line (CR+LF) separates between the Response Headers and Response Body

HTTP Request & Response

```
POST /api/employees HTTP/1.1
Host: localhost:8088
Accept: application/json
Content-Type: application/json

Request Body, If Exists !!!
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 7
Date: Fri, 08 Apr 2022 20:00:39 GMT

Response Body, If Exists !!!
```

HTTP Methods

- HTTP defines a set of ***request methods*** to indicate
  the desired action to be performed for a given resource.

- Although they can also be nouns,
  these request methods are sometimes referred to as HTTP verbs.

- Each of them implements a different semantic,
  but some common features are shared by a group of them:
  e.g., a request method can be ***safe***, ***idempotent***, or ***cacheable***.

- Detailed Explanation
  - https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods

HTTP Methods

Method	Description
GET	Requests a representation of the specified resource. Requests using GET should only retrieve data.
HEAD	Asks for a response identical to a GET request, but without the response body.
POST	Submits an entity to the specified resource, often causing a change in state or side effects on the server.
PUT	Replaces all current representations of the target resource with the request payload.
DELETE	Deletes the specified resource.
CONNECT	Establishes a tunnel to the server identified by the target resource.
OPTIONS	Describes the communication options for the target resource.
TRACE	Performs a message loop-back test along the path to the target resource.
PATCH	Applies partial modifications to a resource.

Safe (HTTP Methods)

- An HTTP method is safe if it doesn't alter the state of the server.
- In other words, a method is safe if it leads to a read-only operation.
- Several common HTTP methods are safe: GET, HEAD, or OPTIONS.

- All safe methods are also idempotent, but not all idempotent methods are safe.
- For example, PUT and DELETE are both idempotent but unsafe.

Safe (HTTP Methods)

- Even if safe methods have a read-only semantic, servers can alter their state:
  e.g., they can log or keep statistics.

- What is important here is that by calling a safe method,
  the client doesn't request any server change itself, and therefore,
  won't create an unnecessary load or burden for the server.

- Browsers can call safe methods without fearing to cause any harm to the server;
  this allows them to perform activities like pre-fetching without risk.

- Web crawlers also rely on calling safe methods.

Safe (HTTP Methods)

- Safe methods don't need to serve static files only;
  a server can generate an answer to a safe method on-the-fly,
  as long as the generating script guarantees safety:
  it should not trigger external effects,
  like triggering an order in an e-commerce Web site.

- It is the responsibility of the application on the server to implement the safe semantic correctly, the webserver itself, being Apache, Nginx or IIS, can't enforce it by itself.

- In particular, an application should not allow GET requests to alter its state.

Safe (HTTP Methods)

- A call to a safe method, not changing the state of the server:
  - GET /pageX.html HTTP/1.1
- A call to a non-safe method, that may change the state of the server:
  - POST /pageX.html HTTP/1.1
- A call to an idempotent but non-safe method:
  - DELETE /idX/delete HTTP/1.1

Idempotent (HTTP Methods)

- An HTTP method is idempotent if an identical request can be made once or several times in a row with the same effect while leaving the server in the same state.

- In other words, an idempotent method should not have any side-effects (except for keeping statistics).

- Implemented correctly, the GET, HEAD, PUT, and DELETE methods are idempotent, but not the POST method.

- All safe methods are also idempotent.

Idempotent (HTTP Methods)

- To be idempotent, only the actual back-end state of the server is considered, the status code returned by each request may differ: the first call of a DELETE will likely return a 200, while successive ones will likely return a 404.

- Another implication of DELETE being idempotent is that developers should not implement RESTful APIs with a delete last entry functionality using the DELETE method.

- Note that the idempotence of a method is not guaranteed by the server and some applications may incorrectly break the idempotence constraint.

Idempotent (HTTP Methods)

- GET /pageX HTTP/1.1 is **idempotent**. Called several times, the client gets the same results:
  - GET /pageX HTTP/1.1
  - GET /pageX HTTP/1.1
  - GET /pageX HTTP/1.1
- POST /add_row HTTP/1.1 is **not idempotent**; if it is called several times, it adds several rows:
  - POST /add_row HTTP/1.1
  - POST /add_row HTTP/1.1 -> Adds a 2nd row
  - POST /add_row HTTP/1.1 -> Adds a 3rd row
- DELETE /idX/delete HTTP/1.1 is **idempotent**, even if the returned status code changed:
  - DELETE /idX/delete HTTP/1.1 -> Returns 200 if idX exists
  - DELETE /idX/delete HTTP/1.1 -> Returns 404 as it just got deleted
  - DELETE /idX/delete HTTP/1.1 -> Returns 404

HTTP Versions

- HTTP/0.9 – The one-line protocol
  - Extremely simple: requests consisted of a single line
- HTTP/1.0 – Building extensibility
  - HTTP/0.9 was very limited, but browsers and servers quickly made it more versatile
- **HTTP/1.1** – The standardized protocol
  - Published in early 1997, only a few months after HTTP/1.0
- HTTP/2 – A protocol for greater performance
  - Binary protocol, Parallel requests can be made over the same connection, compresses headers
- HTTP/3 - HTTP over QUIC
  - Experimental, will use QUIC instead TCP/TLS for the transport layer portion

- Detailed Explanation
  - https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Evolution_of_HTTP

HTTP Headers

- HTTP headers let the client and the server pass additional information with an HTTP request or response.
- An HTTP header consists of its **case-insensitive name** followed by a colon (**:**), then by its **value**.
- Whitespace before the value is ignored.
- Detailed Explanation
  - https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers

HTTP Headers

- Headers can be grouped according to their contexts:

- ***Request headers***
  - contain more information about the resource to be fetched, or about the client requesting the resource.
- ***Response headers***
  - hold additional information about the response, like its location or about the server providing it.
- ***Representation headers***
  - contain information about the body of the resource, like its MIME type or encoding/compression applied.
- ***Payload headers***
  - contain representation-independent information about payload data, including content length and the encoding used for transport.

HTTP Status Codes

- HTTP response status codes indicate
  whether a specific HTTP request has been successfully completed.

- Responses are grouped in five classes:
  1) **Informational responses** (100–199)
  2) **Successful responses** (200–299)
  3) **Redirection messages** (300–399)
  4) **Client error responses** (400–499)
  5) **Server error responses** (500–599)

- Detailed Explanation
  - https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

References

- ***HTTP Tutorials From MDN***
  - https://developer.mozilla.org/en-US/docs/Web/HTTP
- ***Recommended Reading***
  - Cross-Origin Resource Sharing (CORS)
    - https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
  - HTTP authentication
    - https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication
  - HTTP caching
    - https://developer.mozilla.org/en-US/docs/Web/HTTP/Caching
  - Compression in HTTP
    - https://developer.mozilla.org/en-US/docs/Web/HTTP/Compression
  - Content negotiation
    - https://developer.mozilla.org/en-US/docs/Web/HTTP/Content_negotiation
  - Using HTTP cookies
    - https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies